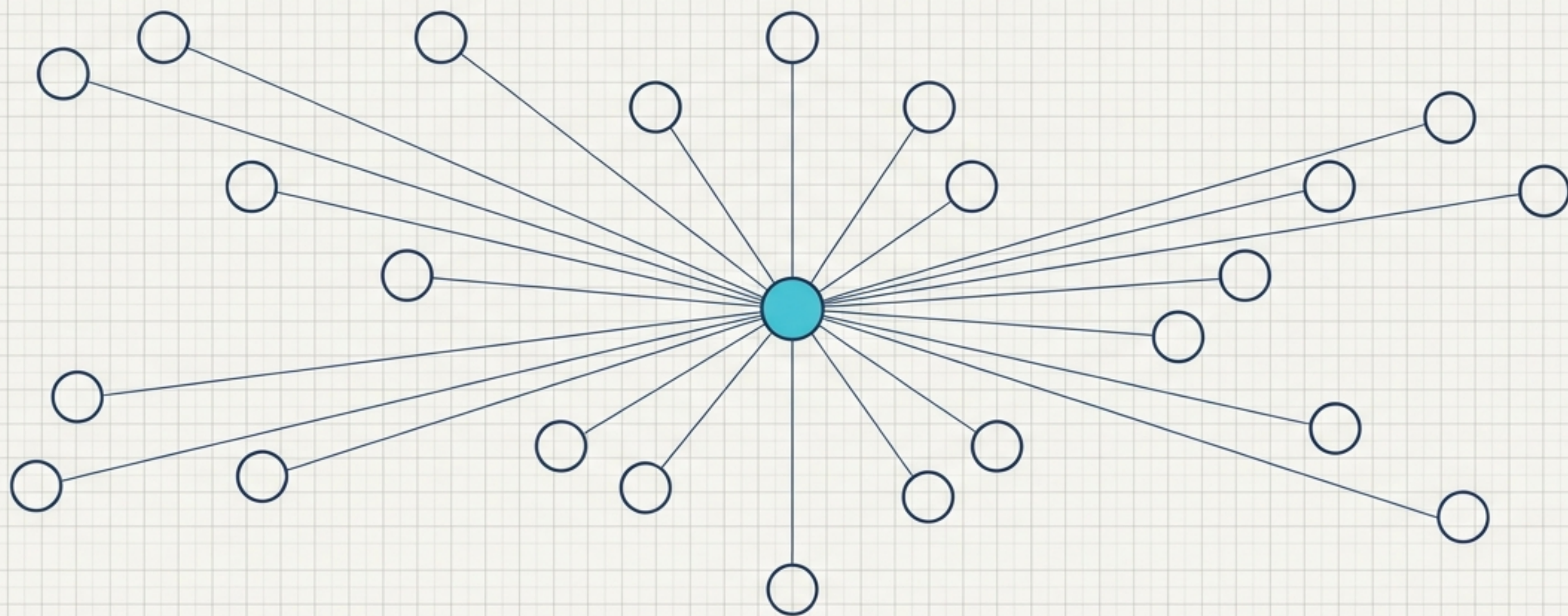


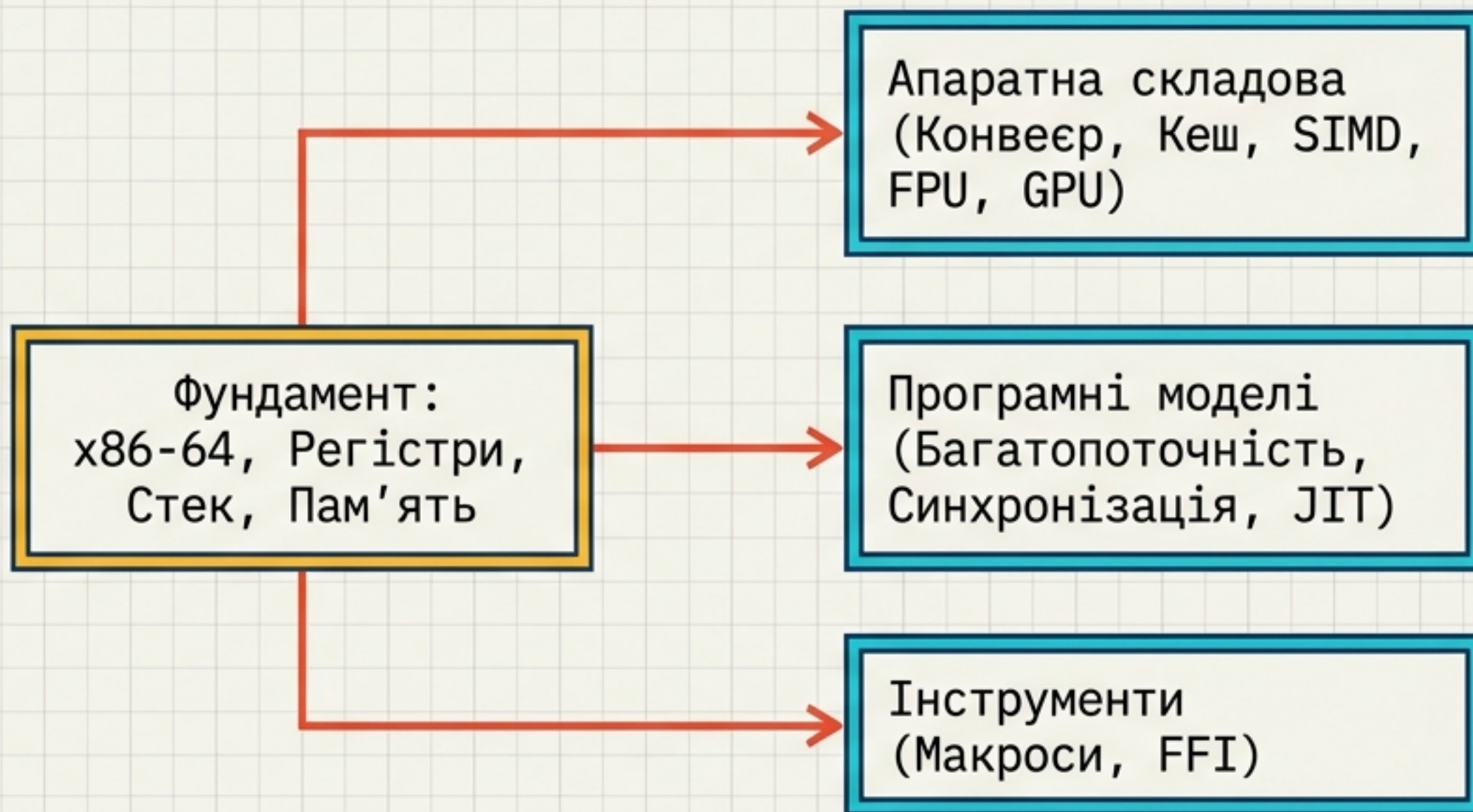
Лекція 15. За межами фундаменту: Карта низькорівневого програмування

Від мікроархітектури процесора до програмних екосистем



Карта тем: Розширюючи межі відомого

Чому ці теми залишилися поза базовим курсом? Деякі вимагають міцного апаратного підґрунтя (вивчати SIMD без знання базових регістрів – це будувати дах без стін), а інші, як багатопоточність чи JIT, заслуговують на окремі курси.



Розуміння низького рівня робить вас кращим програмістом на високому рівні. Навіть код на Python зрештою виконується на процесорі.

Архітектурні філософії: RISC проти CISC

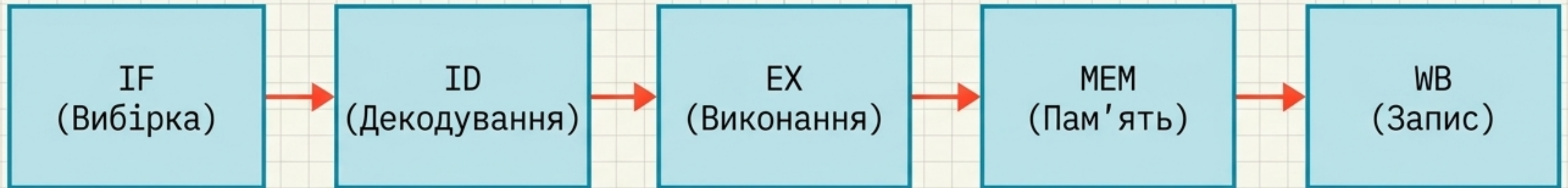
Критерій	CISC	RISC
Кількість інструкцій	Сотні та тисячі	Десятки-сотні
Довжина інструкцій	Змінна	Фіксована
Тактів на інструкцію	1-20 і більше	Переважно 1
Регістри	Менше (16 у x86-64)	Більше (31 у ARM)
Складність декодера	Висока	Низька
Відповідальність	На апаратурі	На компіляторі
Приклади	x86	ARM, RISC-V

Сучасний стан x86



Сучасний x86-процесор є CISC ззовні та RISC зсередини. Трансляція у мікрооперації відбувається апаратно.

Конвеєр виконання інструкцій: Фабрика всередині процесора



⚠ Структурні конфлікти

Брак ресурсів.
Вирішується розділенням
L1i та L1d кешів.

⚠ Конфлікти даних

Залежність від
попереднього результату.
Вирішується пересиланням
(forwarding).

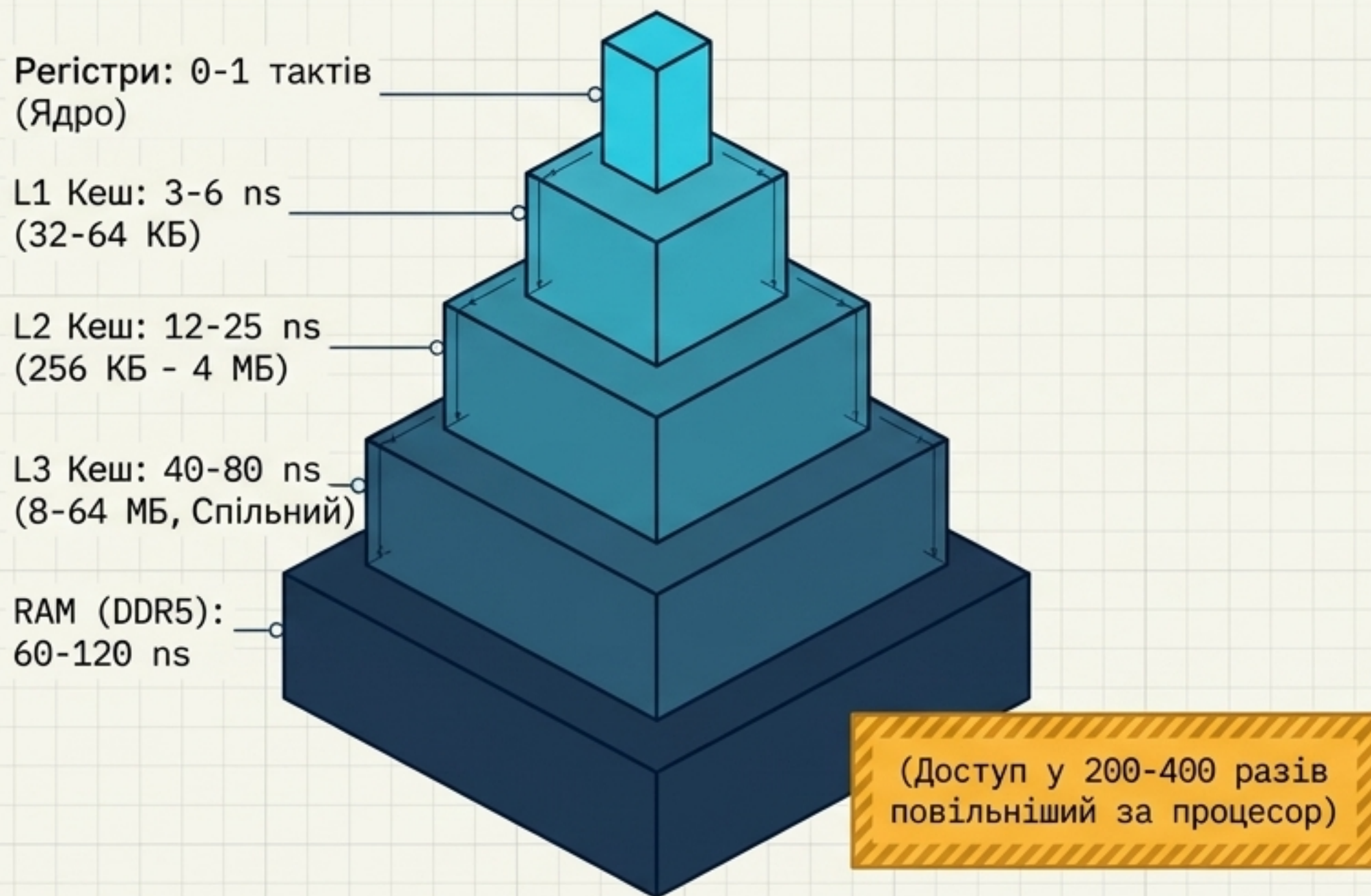
⚠ Конфлікти управління

Умовні переходи.
Найбільший ризик
скидання конвеєра.

Передбачення переходів (Branch Prediction) вгадує результат із точністю 95–99%.
При помилці – скидання конвеєра (штраф 14–20 тактів у сучасних CPU).

Ієрархія пам'яті: Подолання прірви швидкості

Пам'ять оперує кеш-лініями (64 байти). Ефективність покладається на просторову та часову локальність.



Когерентність кешів (MESI)



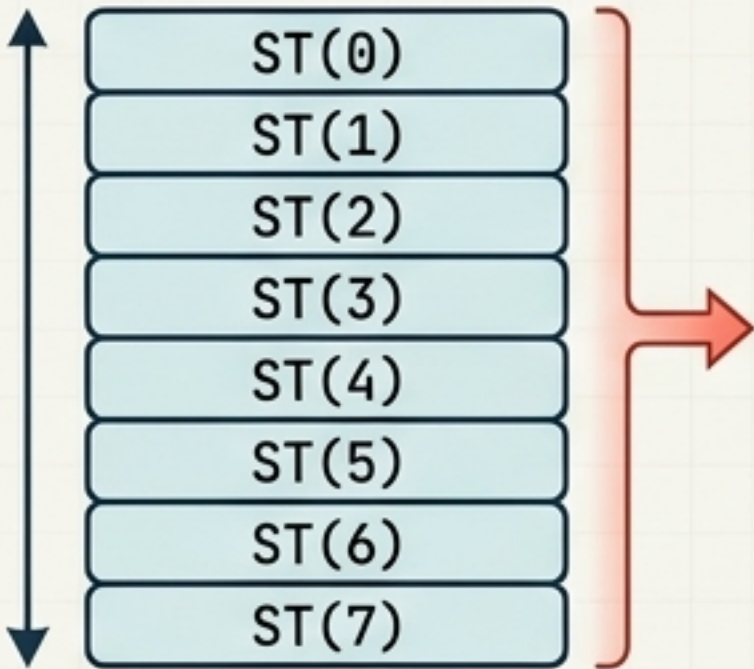
FPU: Обчислення з плаваючою комою та стандарт IEEE 754

ПОРІВНЯННЯ ФОРМАТІВ IEEE 754

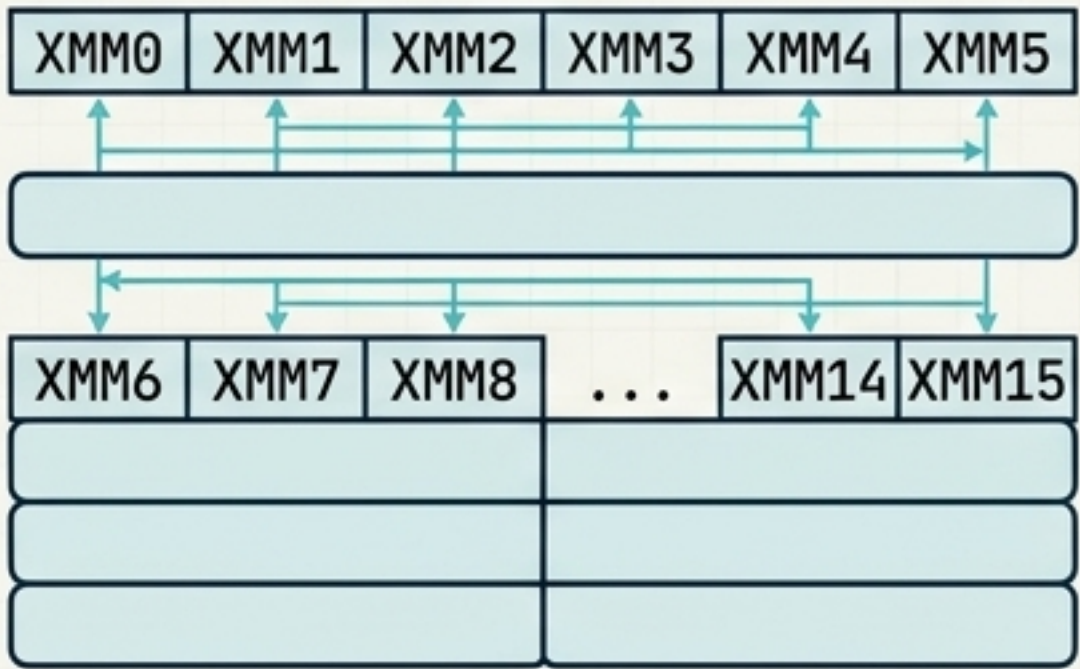
Формат	Розмір	Знак	Показник	Мантиса	Точність
Single	32 біти	1	8	23	~7 знаків
Double	64 біти	1	11	52	~15-17 знаків
Extended	80 бітів	1	15	64	~18-19 знаків

АРХІТЕКТУРИ FPU: ПОРІВНЯННЯ

x87 STACK MODEL



СУЧАСНІ SSE XMM РЕГІСТРИ



⚠ ПОПЕРЕДЖЕННЯ: СПЕЦІАЛЬНІ ЗНАЧЕННЯ

Спеціальні значення: NaN (Not a Number), Нескінченність, Денормалізовані.
NaN 'заражає' всі подальші обчислення.

⚠ ПАСТКА: ТОЧНІСТЬ ТА ОКРУГЛЕННЯ

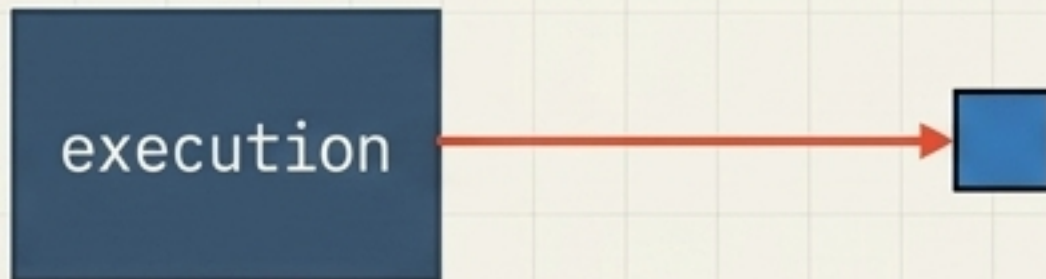
Пастка: $0.1 + 0.2 \neq 0.3$. Накопичення похибки округлення. Порівняння на точну рівність — фундаментальна помилка.

SIMD: Множення сили через паралелізм даних

SISD (Одна інструкція, Одні дані) Класичний x86	SIMD (Одна інструкція, Множинні дані) SSE, AVX
MISD (Множинні інструкції, Одні дані) Теоретичний	MIMD (Множинні інструкції, Множинні дані) Багатоядерний CPU

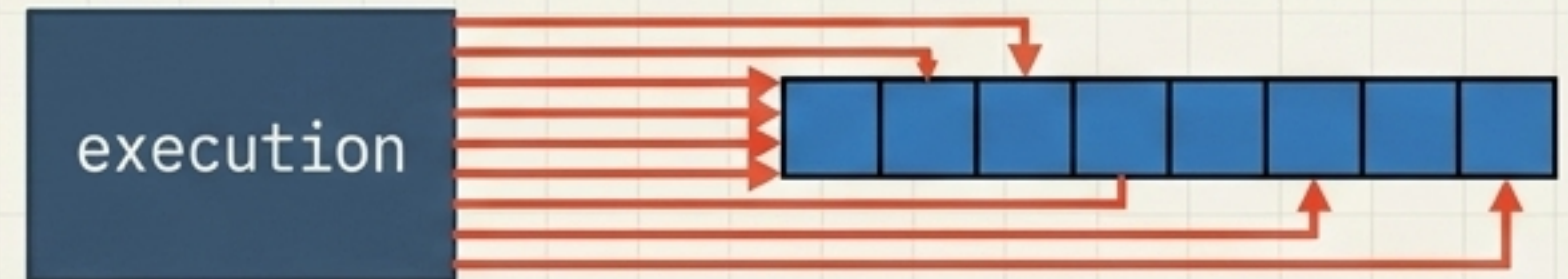
SISD vs SIMD: Скалярна та Векторна Обробка

SISD



1 скалярна інструкція = 1 піксель

SIMD



1 векторна інструкція = 8 пікселів одночасно

Навіщо це потрібно? Обробка відео або ML вимагає мільйонів операцій.

Автовекторизація: Сучасні компілятори самі перетворюють прості цикли у SIMD-код.

Еволюція SIMD (Частина 1): Від MMX до ери SSE

1996

MMX. 64-бітні реєстри MM0-MM7.
Конфлікт із x87 FPU -> необхідність команди EMMS.

HAZARD:
EMMS required

1999

SSE. 128-бітні XMM0-XMM7.
Float SIMD, незалежні від FPU.

BREAKTHROUGH:
FPU Independence

2001

SSE2. Double та цілочисельні типи в XMM.
Базова лінія для сучасного x86-64.

FOUNDATION:
Modern x86-64 Baseline

2004-2008

SSE3 - SSE4.2. Горизонтальні операції, маніпуляції з байтами, CRC-32, рядки.

EVOLUTION:
Specialized Instructions

Практичне застосування:
Стандарт відео H.264 (x264) та FFmpeg інтенсивно використовують SSE для DCT-перетворень та пошуку руху.

Еволюція SIMD (Частина 2): AVX та векторні гіганти

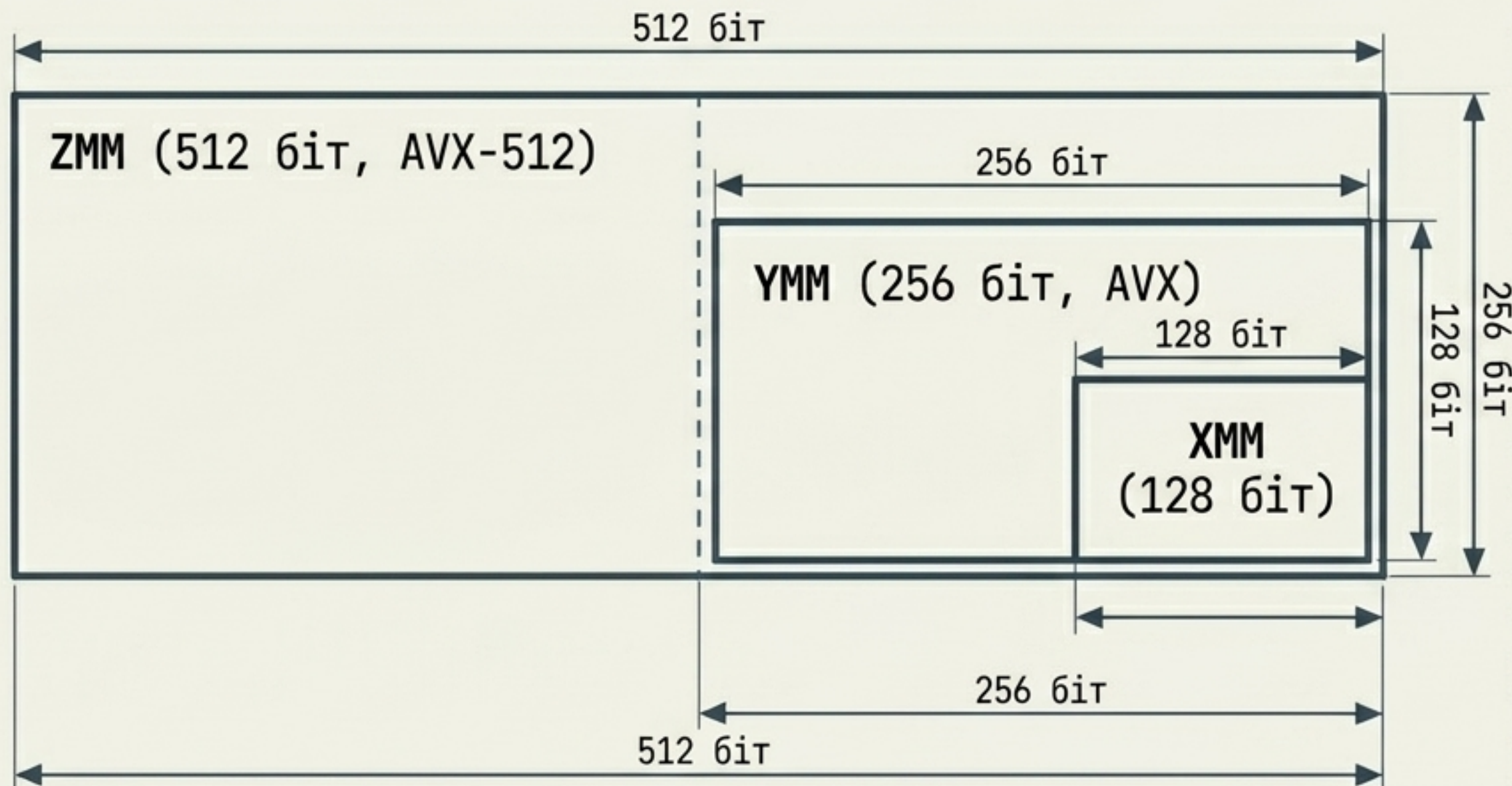
AVX (2011):

Недеструктивне
VEX-кодування.

FMA (Fused Multiply-Add)
для обчислення $a * b + c$.

AVX-512 (2016):

Вводить `opmask` (k0-k7)
для виконання без
розгалужень та
Scatter/Gather для
розріджених даних.

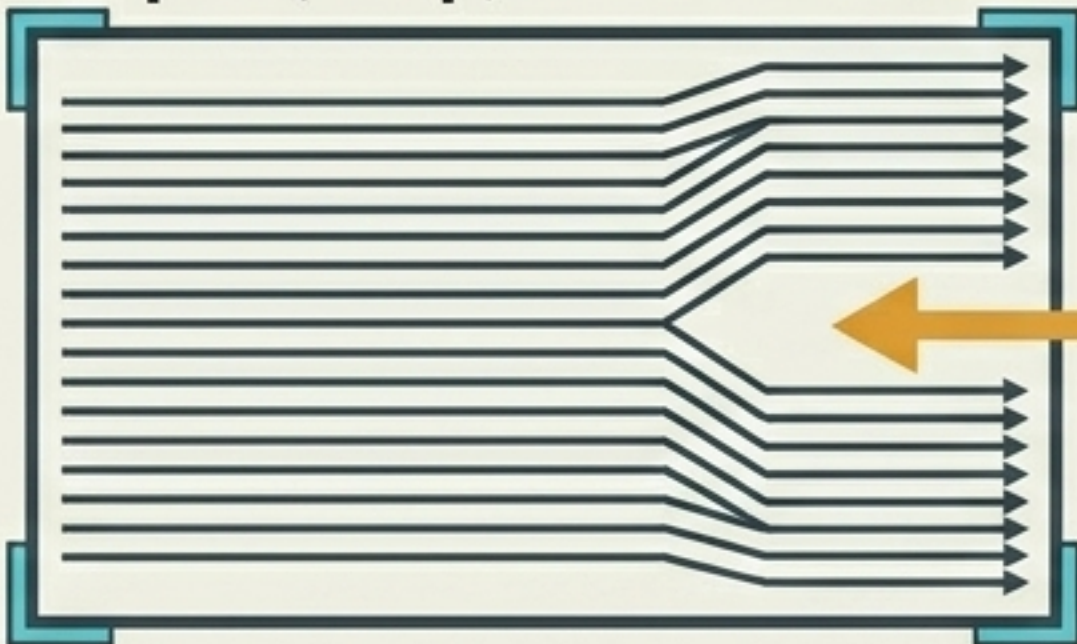


Парадокс зниження частоти: 512-бітні блоки генерують настільки багато тепла, що процесор змушений агресивно знижувати тактову частоту (Thermal Throttling) під час виконання AVX-512 інструкцій.

GPU: Масивно-паралельна архітектура

Характеристика	CPU	GPU
Кількість ядер	8–128	Тисячі (до 16,384 у RTX 4090)
Складність ядра	Дуже висока	Низька
Оптимізація	Мала затримка (Latency)	Висока пропускна здатність (Throughput)

Варп (Warp) = 32 потоки



Warp Divergence
(последовное
выполнение
веток if/else)

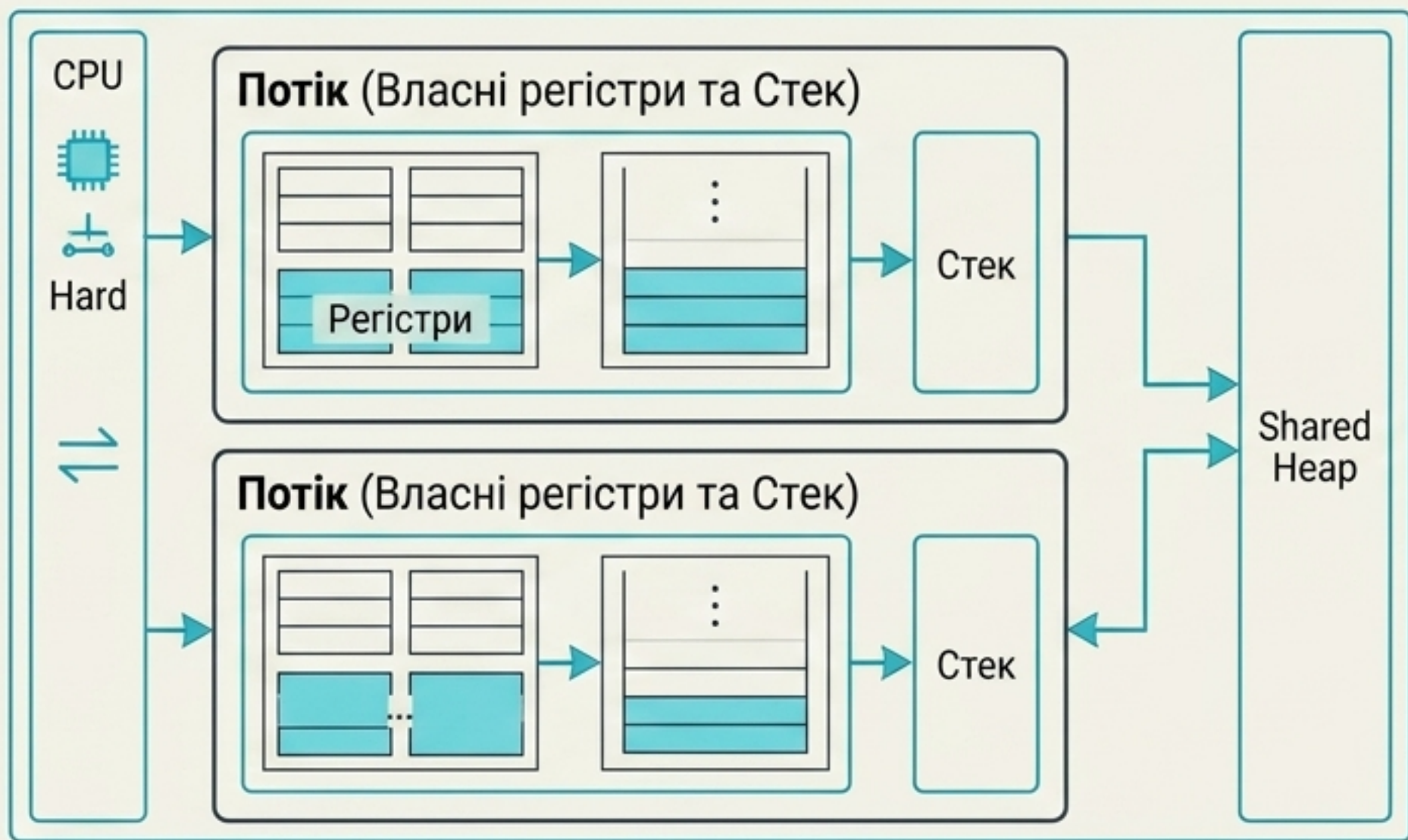
SIMT (Single Instruction,
Multiple Threads).

Пропускна здатність пам'яті
(VRAM) на H100 сягає 3.35 ТБ/с
(в 10 разів більше за DDR5).

GPGPU (CUDA) відкрив шлях для
тренування неймереж.

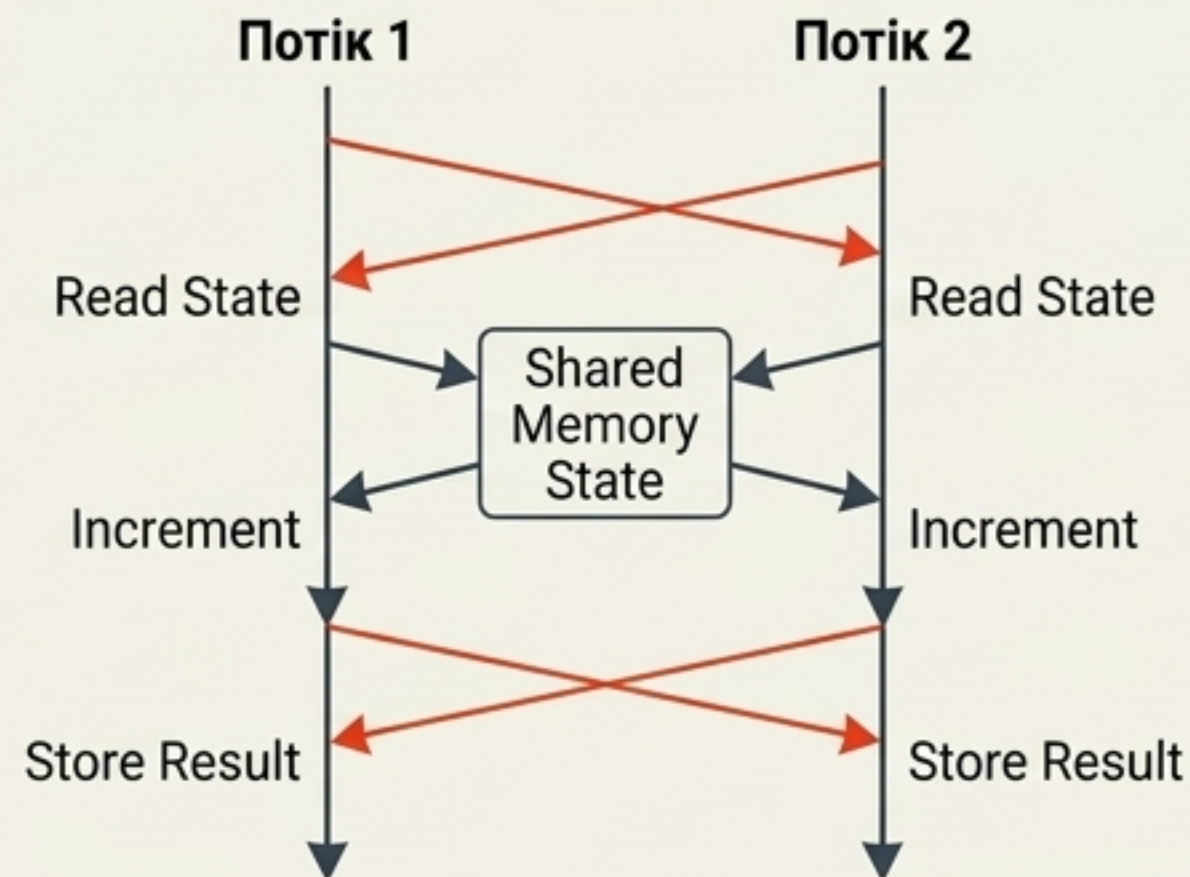
Багатопоточність: Програмне управління паралелізмом

Процес (Ізольований адресний простір: Shared Heap, Code, Globals)

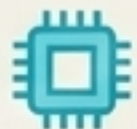


Mechanism Panel

Data Race Anatomy



Результат: Втрачене оновлення (Data Race)

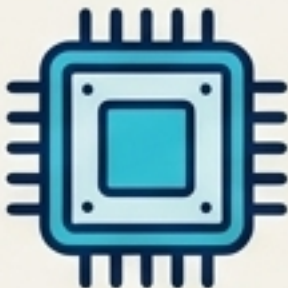


Hyper-Threading (SMT): Два логічних потоки ділять одне фізичне ядро.



Пул потоків (Thread Pool): Створення потоку дороге. Пул тримає потоки готовими до завдань, мінімізуючи системні виклики ОС.

Синхронізація: Захист критичних секцій

1	2	3
 Апаратний рівень Префікс <code>`LOCK`</code> блокує шину пам'яті. <code>`LOCK CMPXCHG`</code> реалізує Compare-And-Swap (CAS). Бар'єри пам'яті (<code>MFENCE`</code>) забороняють переупорядкування інструкцій.	 Mutex & Spinlocks Взаємне виключення для критичних секцій. Потік переходить у сон (Mutex) або активно чекає в циклі (Spinlock).	 Семафор (Semaphore) Лічильник ресурсів. Операції <code>`wait`</code> (зменшує) та <code>`signal`</code> (збільшує).



Дедлок (Deadlock): Теорема Коффмана описує 4 умови виникнення нескінченного взаємного очікування. Найпростіше рішення: завжди захоплювати Mutex-и в однаковому строгому порядку.

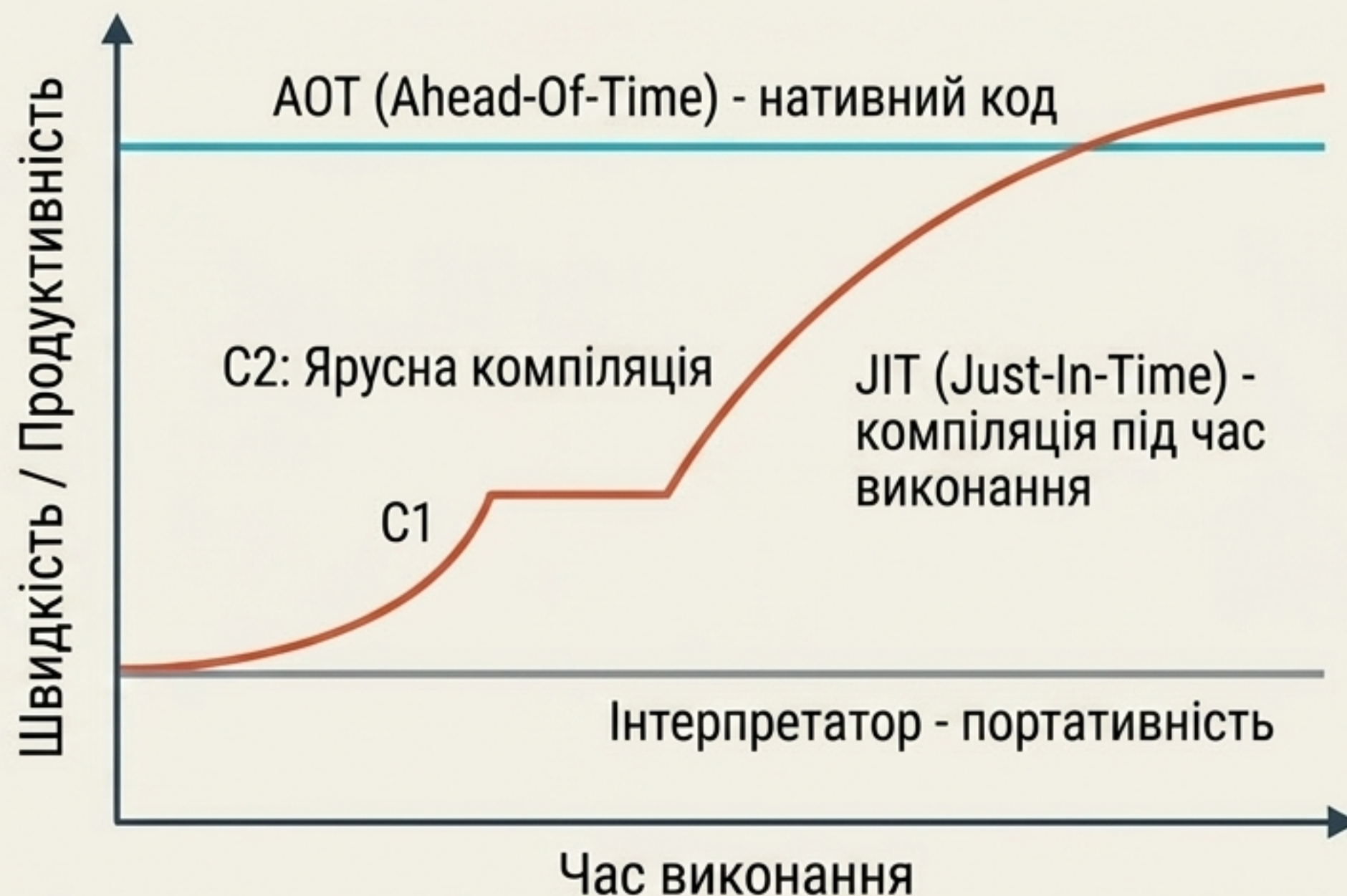
JIT-компіляція: Код, що оптимізує сам себе

Проміжне представлення:

Вихідний код перетворюється у Байткод (Java CIL, .NET) перед виконанням.

Ярусна компіляція (Hotspot):

C1 (швидка генерація) -> C2 (агресивні оптимізації гарячого коду).



Час прогріву та Деоптимізація: Спекулятивні припущення JIT можуть бути хибними (напр. зміна типу аргументу). У цьому разі нативний код відкидається. Приклади: V8 (Chrome), PyPy.

Макроасемблер: Текстові підстановки без накладних витрат

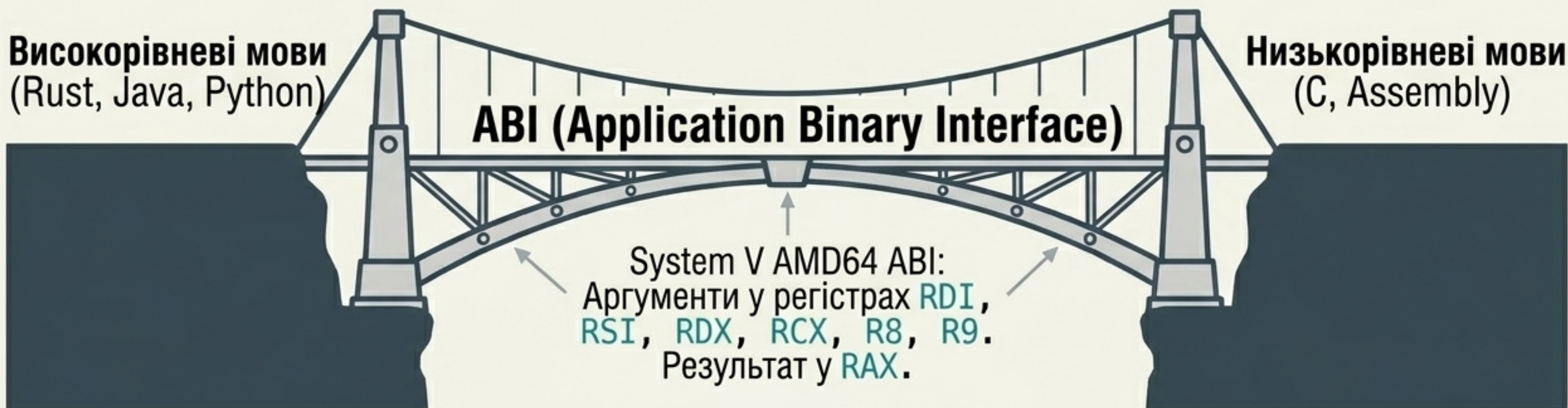
Макрос — це генерація тексту ДО асемблювання. Відсутній стек викликів, інструкції call/ret та збереження регістрів.

Повторюваний Assembly	NASM Макрос
<pre>push rax push rbx pop rbx pop rax push rax push rbx pop rbx pop rax push rax push rbx pop rbx pop rax push rax push rbx pop rbx pop rax</pre>	<pre>%ifdef USE_MACRO %macro PUSHPOP 0 push rax push rbx pop rbx pop rax %endmacro %rep 4 PUSHPOP %endrep %endif</pre>  (Генерує 4 блоки інструкцій push/pop)

⚠ Компроміс: Швидкість проти розміру

Макроси прискорюють виконання дрібних операцій, але багаторазове розгортання збільшує фізичний розмір бінарного файлу. Це може призвести до промахів у L1-кеші інструкцій. (Аналог — C-препроцесор).

Комбінування мов: Мости через рівні абстракції



Rust FFI:

Блоки `extern 'C'` та `unsafe`.

Java Native Interface (JNI):

Вимагає об'єкта `JNIEnv` для взаємодії з віртуальною машиною `JVM`.

Python / Node.js:

`ctypes`, `cffi`, `N-API`.



Міжмовні виклики не безкоштовні (перетворення типів, обгортання). Найдоцільніші для 'батч-операцій'.

Підсумок: Рівень абстракції зростає, фундамент залишається

	Тема	Базовий ресурс	Поглиблений ресурс
1	Архітектура	Patterson & Hennessy	Computer Architecture: Quantitative
2	Кеш / Пам'ять	CS:APP, глава 6	Drepper: What Every Programmer Should Know
3	SIMD / Оптимізація	Intel Intrinsics Guide	Agner Fog: Microarchitecture
4	GPU	CUDA Programming Guide	Massively Parallel Processors
5	Багатопоточність	POSIX pthreads	C++ Concurrency in Action

Final Takeaway

Кожен цикл залежить від **кешу**. Кожна багатопоточна гонка підпорядковується **апаратній пам'яті**. Знання **низького рівня** — це ваша беззаперечна перевага при створенні **системного ПЗ**, **компіляторів**, **GameDev рушіїв** та **безпекового софту**. Карта відкрита — шлях за вами.